Chuyên đề: Tìm kiếm nhị phân & Tìm kiếm nhị phân trên tập kết quả

I. Lý thuyết nền tảng

1. Bản chất của Tìm kiếm nhị phân (Binary Search)

Tìm kiếm nhị phân không chỉ đơn thuần là tìm một phần tử trong mảng đã sắp xếp. Bản chất cốt lõi của nó là **thu hẹp không gian tìm kiếm** dựa trên tính đơn điệu (monotonicity) của một hàm số hoặc một tính chất logic.

Giả sử ta có một hàm logic $P(x)$ trả về giá trị $True$ hoặc $False$. Nếu $P(x)$ có tính chất:

- Nếu $P(x) = True$, thì với mọi $y > x$, $P(y)$ cũng bằng $True$.
- Hoặc ngược lại (tính đơn điệu giảm).

Thì ta hoàn toàn có thể dùng Binary Search để tìm điểm chuyển giao (boundary) giữa $True$ và $False$ trong độ phân giải thời gian $O(\log N)$.

2. Tìm kiếm nhị phân trên tập số nguyên

Cấu trúc phổ biến nhất để tránh các lỗi lặp vô hạn (infinite loop) hoặc bỏ sót nghiệm là sử dụng hai biến L, R và tính $mid = L + \frac{R-L}{2}$.

- **Tìm giá trị nhỏ nhất thỏa mãn điều kiện $P(x)$:** Nếu $P(mid)$ đúng, nghiệm có thể là mid hoặc nằm bên trái $mid \to R = mid - 1$. Ngược lại, $L = mid + 1$.
- **Tìm giá trị lớn nhất thỏa mãn điều kiện $P(x)$:** Nếu $P(mid)$ đúng, nghiệm có thể là mid hoặc nằm bên phải $mid \to L = mid + 1$. Lưu ý cách tính $mid = L + \frac{R-L+1}{2}$ để tránh loop khi $R = L + 1$.

3. Tìm kiếm nhị phân trên tập số thực

Đối với số thực, ta không thể sử dụng $L = mid + 1$. Thay vào đó, ta thực hiện lặp một số lần cố định (thường là 100 lần) để đạt độ chính xác cực cao, hoặc dùng sai số ϵ:

$$|R - L| > \epsilon$$

II. Các bài toán điển hình và Phương pháp tiếp cận chuyên sâu

1. Tìm vị trí đầu tiên có giá trị lớn hơn x (Upper Bound)

Bài toán: Cho mảng A gồm N phần tử đã sắp xếp tăng dần. Với mỗi truy vấn x, tìm vị trí (1-indexed) của phần tử đầu tiên thỏa mãn $A[i] > x$. Nếu không tồn tại, in ra -1.

Quy trình tư duy & Mổ xẻ thuật toán:

- **Tính đơn điệu (Monotonicity):** Vì mảng A đã sắp xếp tăng dần, nên nếu tại vị trí i ta có $A[i] > x$, thì chắc chắn với mọi $j > i$, $A[j] > x$. Mảng kết quả kiểm tra điều kiện sẽ có dạng: [False, False, ..., False, **True**, True, ...]. Nhiệm vụ của ta là tìm chữ **True** đầu tiên.
- **Thiết lập không gian tìm kiếm:** Khởi tạo $L = 1, R = N$. Ta dùng một biến $ans = -1$ để ghi nhận trạng thái hợp lệ tốt nhất đã từng gặp.
- **Cốt lõi xử lý (Loop Invariant):**
  - Khi $A[mid] > x$: Vị trí mid thỏa mãn điều kiện. Ta cập nhật $ans = mid$. Tuy nhiên, có thể vẫn còn phần tử nằm bên trái mid cũng lớn hơn x, do đó ta phải thu hẹp không gian tìm kiếm sang trái bằng cách gán $R = mid - 1$.
  - Khi $A[mid] \leq x$: Vị trí mid không thỏa mãn, và mọi vị trí bên trái mid cũng vậy. Ta bắt buộc phải tìm ở nửa phải bằng cách gán $L = mid + 1$.
- **Độ phức tạp:** Thời gian $O(\log N)$ cho mỗi truy vấn, Không gian bộ nhớ $O(1)$.

Listing 1: Cài đặt Upper Bound thủ công

```
int solve(int x) {
    int L = 1, R = n, ans = -1;
    while (L <= R) {
        // Dung L + (R-L)/2 thay vi (L+R)/2 de tranh tran so (Integer
   Overflow)
        int mid = (R + L) / 2;
        if (a[mid] > x) {
            ans = mid;      // Ghi nhan ket qua tam thoi
            R = mid - 1;    // Tiep tuc ep ve phia ben trai
        } else {
            L = mid + 1;    // Vut bo nua ben trai khong thoa man
        }
    }
    return ans;
}
```

2. Bài toán Cắt Gỗ (Woodcut - Tối ưu hóa tài nguyên)

Bài toán: Có N thanh gỗ, thanh thứ i có độ dài L_i. Bạn cần cắt các thanh gỗ này thành đúng K đoạn có cùng chiều dài H (H nguyên dương). Các mẩu gỗ thừa sẽ bị vứt bỏ. Tìm chiều dài H lớn nhất có thể.

Quy trình tư duy & Mổ xẻ thuật toán:

- **Tư duy nghịch đảo:** Thay vì cố gắng giải phương trình trực tiếp để tìm H từ các đoạn L_i (rất phức tạp vì hàm làm tròn xuống), ta "đoán" kết quả H và kiểm tra tính khả thi.
- **Tính đơn điệu:** Nếu ta có thể cắt được $\geq K$ khúc gỗ độ dài H, thì ta chắc chắn cắt được $\geq K$ khúc gỗ với độ dài $H' < H$. Ngược lại, nếu với độ dài H mà không đủ K khúc, thì mọi độ dài lớn hơn H cũng sẽ thất bại. Dãy logic có dạng: [True, ..., **True**, False, ...]. Bài toán yêu cầu tìm **True** cuối cùng.

- **Hàm `Check(H)`:** Với một độ dài H cho trước, một thanh gỗ L_i sẽ cung cấp tối đa $\lfloor L_i/H \rfloor$ khúc gỗ. Ta tính tổng số khúc gỗ cắt được từ N thanh và so sánh với K.
- **Biên tìm kiếm:** $L = 1$, $R = \max(L_1, L_2, \ldots, L_n)$.

Listing 2: Cài đặt thuật toán Cắt Gỗ

```
bool check(long long H, long long K) {
    long long count = 0;
    for (int i = 1; i <= n; i++) {
        count += (L[i] / H);
    }
    return count >= K;
}

long long solve_woodcut(long long K) {
    long long low = 1, high = 0, ans = 0;
    for (int i = 1; i <= n; i++) high = max(high, L[i]); // Tim R lon nhat

    while (low <= high) {
        long long mid = (high + low) / 2;
        if (check(mid, K)) {
            ans = mid;        // H thoa man, luu lai ket qua
            low = mid + 1;    // Co gang tim H lon hon nua
        } else {
            high = mid - 1;   // H qua lon, khong du K khuc, phai giam H
        }
    }
    return ans;
}
```

3. Tìm kiếm nhị phân trên tập số thực: Tính căn bậc 3 của số S

Bài toán: Cho số thực S ($|S| \leq 10^{18}$). Tìm x sao cho $x^3 = S$ với sai số không vượt quá 10^{-7}.

Quy trình tư duy & Mổ xẻ thuật toán:

- **Tính đơn điệu:** Hàm $f(x) = x^3$ là hàm đồng biến chặt trên toàn trục số thực $\mathbb{R}$. Do đó chỉ tồn tại duy nhất một nghiệm x.
- **Thiết lập biên:** Do số thực có thể $\in (-1, 1)$, ta không thể thiết lập $L = 0, R = S$. Cách an toàn nhất cho các bài CP là lấy biên giới hạn của đề bài. Ví dụ $S \leq 10^{18} \implies x \leq 10^6$. Ta khởi tạo $L = -10^6, R = 10^6$.
- **Kỹ thuật vòng lặp số thực:** Thay vì dùng ‘while (R - L > eps)‘, cách làm này cực kỳ dễ sinh ra lỗi lặp vô hạn (Infinite Loop) do giới hạn độ chính xác bit của kiểu ‘double‘. Việc lặp cố định 100 lần là "tiêu chuẩn vàng" trong OI, giúp chia đôi khoảng không gian 100 lần. $2^{100} \approx 10^{30}$, đủ sức bóp nát mọi khoảng sai số để cho ra kết quả chính xác tuyệt đối.

Listing 3: Kỹ thuật lặp 100 lần cho số thực

```
double solve_cbrt(double S) {
```

```
    double L = -1e6, R = 1e6; // Bien an toan
    for (int i = 0; i < 100; i++) {
        double mid = (R + L) / 2.0;
        if (mid * mid * mid <= S) {
            L = mid; // Khong co mid + 1 voi so thuc
        } else {
            R = mid; // Khong co mid - 1
        }
    }
    return L; // Sau 100 vong lap, L va R gan nhu bang nhau
}
```

4. Khoảng cách lớn nhất giữa K điểm (Aggressive Cows)

Bài toán: Cho một trục tọa độ thẳng và N vị trí x_i có thể đặt vật ($N \leq 10^5$). Bạn cần chọn ra đúng K vị trí để đặt K đồ vật sao cho **khoảng cách ngắn nhất** giữa 2 vật bất kỳ là **lớn nhất** có thể.

Quy trình tư duy & Mổ xẻ thuật toán:

- **Nhận diện dạng bài:** "Minimizing the Maximum" (Cực tiểu hóa giá trị lớn nhất) hoặc "Maximizing the Minimum" (Cực đại hóa giá trị nhỏ nhất) là dấu hiệu kinh điển của Chặt nhị phân kết quả.
- **Hàm `Check(D)`:** Giả sử ta muốn khoảng cách tối thiểu giữa mọi cặp vật là D. Liệu ta có thể đặt được ít nhất K vật không?
- **Chiến thuật Tham lam (Greedy Choice Property):**
  - **Bắt buộc sắp xếp mảng tọa độ tăng dần.**
  - Vật đầu tiên luôn luôn đặt ở x_1. Càng đặt lệch sang phải, không gian còn lại cho $K-1$ vật kia càng hẹp lại.
  - Vật tiếp theo sẽ được đặt ở vị trí x_i *gần nhất* tính từ vật trước đó thỏa mãn x_i – vị trí trước $\geq D$. Điều này nhằm tiết kiệm không gian tối đa cho các vật sau.

Listing 4: Chiến thuật Tham Lam kết hợp Chặt nhị phân

```
bool check(int D, int K, int n, int x[]) {
    int placed = 1;          // Dat vat dau tien
    int last_pos = x[1];     // Vi tri vat truoc do la x[1]

    for (int i = 2; i <= n; i++) {
        if (x[i] - last_pos >= D) {
            placed++;            // Dat duoc them 1 vat
            last_pos = x[i];     // Cap nhat lai toa do cua vat moi
   nhat
            if (placed >= K) return true; // Dat du K vat roi, thanh
   cong!
        }
    }
    return false;
}

int solve_cows(int n, int K, int x[]) {
```

```
    sort(x + 1, x + n + 1); // BUOC QUAN TRONG NHAT: Sap xep toa do

    int low = 1, high = x[n] - x[1], ans = -1;
    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (check(mid, K, n, x)) {
            ans = mid;      // Co the mo rong khoang cach D
            low = mid + 1;
        } else {
            high = mid - 1; // Khong dat duoc K vat, D qua lon, phai thu hep D
        }
    }
    return ans;
}
```

III. Bài tập vận dụng

Bài 1: Truy vấn giới hạn trên (Lower Bound / Last Element $\leq x$)

Thời gian tối đa: 1.0s
Bộ nhớ tối đa: 256MB

Mô tả bài toán:

Cho mảng A không giảm gồm n số nguyên. Cho q truy vấn, mỗi truy vấn là một số nguyên x, yêu cầu tìm chỉ số i **lớn nhất** sao cho $A_i \leq x$.

Input:

- Dòng đầu tiên gồm 2 số nguyên n, q.
- Dòng tiếp theo gồm n số nguyên A_i.
- q dòng tiếp theo, mỗi dòng gồm một số nguyên x, đại diện cho một truy vấn.

Output:

- In ra q dòng, dòng thứ i là kết quả của truy vấn i. Nếu không tìm thấy i nào trong mảng A thỏa mãn, in ra `-1`.

Điều kiện:

- $1 \leq n, q \leq 10^5$.
- $1 \leq A_i \leq 10^9$.

Ví dụ:

Input	Output
5 3 2 4 4 7 9 4 1 8	3 -1 4

Giải thích ví dụ:

- Truy vấn $x = 4$: Có hai vị trí thỏa mãn $A_i \leq 4$ là $i = 1, 2, 3$. Vị trí lớn nhất là 3.
- Truy vấn $x = 1$: Không có phần tử nào ≤ 1, trả về `-1`.
- Truy vấn $x = 8$: Các phần tử ≤ 8 nằm ở $i = 1, 2, 3, 4$. Vị trí lớn nhất là 4.

Bài 2: Đếm số lượng dãy con liên tiếp

Thời gian tối đa: 1.0s
Bộ nhớ tối đa: 256MB

Mô tả bài toán:

Cho mảng A gồm n số nguyên dương. Hãy đếm số lượng dãy con liên tiếp có tổng không nhỏ hơn k.

Input

- Dòng đầu tiên gồm 2 số nguyên n và k.
- Dòng tiếp theo gồm n số nguyên dương A_i cách nhau bởi dấu cách.

Output

- In ra một số nguyên duy nhất là số lượng dãy con liên tiếp có tổng không nhỏ hơn k.

Điều kiện

- $1 \leq n \leq 10^5$.
- $1 \leq A_i \leq 10^9$.
- $1 \leq k \leq 10^{14}$.

Ví dụ

Input	Output
4 6 2 4 1 5	5

Giải thích ví dụ: Có 5 dãy con liên tiếp có tổng ≥ 6 là:

- Dãy gồm phần tử từ vị trí 1 đến 2: $2 + 4 = 6$
- Dãy gồm phần tử từ vị trí 1 đến 3: $2 + 4 + 1 = 7$
- Dãy gồm phần tử từ vị trí 1 đến 4: $2 + 4 + 1 + 5 = 12$
- Dãy gồm phần tử từ vị trí 2 đến 4: $4 + 1 + 5 = 10$
- Dãy gồm phần tử từ vị trí 3 đến 4: $1 + 5 = 6$

Bài 3: Thống kê tần số trong khoảng (Range Frequency Query)

Thời gian tối đa: 1000 ms
Giới hạn bộ nhớ: 256 MB

Mô tả bài toán:

Trong quá trình phân tích chuỗi dữ liệu, hệ thống thu thập được một mảng A bao gồm n số nguyên. Để phục vụ cho việc khai phá thông tin, bạn được giao nhiệm vụ xây dựng một module trả lời q truy vấn từ người dùng.

Cụ thể, mỗi truy vấn sẽ cung cấp cho bạn một bộ ba tham số (l, r, x). Nhiệm vụ của bạn là phải đếm xem giá trị mục tiêu x xuất hiện bao nhiêu lần nếu chỉ xét các phần tử nằm từ vị trí thứ l đến vị trí thứ r (bao gồm cả hai điểm mút) trong mảng A.

Input

- Dòng đầu tiên chứa hai số nguyên dương n và q đại diện cho kích thước mảng dữ liệu và số lượng truy vấn cần xử lý.
- Dòng thứ hai chứa n số nguyên $A_1, A_2, \ldots, A_n$ phân tách nhau bằng một khoảng trắng.
- q dòng tiếp theo, mỗi dòng chứa ba số nguyên l, r, x mô tả các tham số của một truy vấn.

Output

- In ra q dòng, dòng thứ i chứa một số nguyên duy nhất là kết quả thống kê tương ứng cho truy vấn thứ i.

Điều kiện

- $1 \leq n, q \leq 10^5$.
- $1 \leq A_i, x \leq 10^5$.
- $1 \leq l \leq r \leq n$.

Ví dụ

Input	Output
6 3 1 2 2 3 2 1 1 5 2 2 4 1 4 6 1	3 0 1

Giải thích ví dụ:

- **Truy vấn 1 (1, 5, 2):** Xét mảng con từ vị trí 1 đến 5 là `[1, 2, 2, 3, 2]`. Số `2` xuất hiện 3 lần (tại vị trí 2, 3 và 5).
- **Truy vấn 2 (2, 4, 1):** Xét mảng con từ vị trí 2 đến 4 là `[2, 2, 3]`. Số `1` xuất hiện 0 lần.
- **Truy vấn 3 (4, 6, 1):** Xét mảng con từ vị trí 4 đến 6 là `[3, 2, 1]`. Số `1` xuất hiện 1 lần (tại vị trí 6).

Bài 4: Phân loại kẹo trên băng chuyền

Thời gian tối đa: 1.0s
Bộ nhớ tối đa: 256MB

Mô tả bài toán:

Một nhà máy bánh kẹo sản xuất n loại kẹo khác nhau. Quá trình thống kê cho thấy, loại kẹo thứ i được sản xuất với số lượng là a_i chiếc, và mỗi viên kẹo loại này đều có khối lượng chuẩn là w_i. Đảm bảo rằng không có bất kỳ hai loại kẹo nào có cùng khối lượng với nhau.

Sau khi sản xuất, toàn bộ số kẹo được đưa lên một băng chuyền lớn và được hệ thống tự động sắp xếp thành một hàng ngang theo thứ tự khối lượng không giảm (từ nhẹ nhất đến nặng nhất).

Bộ phận kiểm định chất lượng cần thực hiện q truy vấn kiểm tra độc lập. Trong mỗi truy vấn, họ cung cấp một số nguyên k và muốn biết: Viên kẹo nằm ở vị trí thứ k trên băng chuyền có khối lượng là bao nhiêu?

Input

- Dòng đầu tiên chứa hai số nguyên dương n và q.
- n dòng tiếp theo, mỗi dòng chứa hai số nguyên a_i và w_i mô tả số lượng và khối lượng của loại kẹo thứ i.
- q dòng tiếp theo, mỗi dòng chứa một số nguyên k đại diện cho một truy vấn vị trí.

Output

- In ra q dòng, dòng thứ i chứa một số nguyên duy nhất là khối lượng của viên kẹo ở vị trí thứ k tương ứng với truy vấn thứ i.

Điều kiện

- $1 \leq n, q \leq 10^5$.
- $1 \leq a_i, w_i \leq 10^9$.
- $1 \leq k \leq a_1 + a_2 + \cdots + a_n \leq 10^{14}$.

Ví dụ

Input	Output
3 3 2 10 3 5 1 15 1 4 6	5 10 15

Giải thích ví dụ: Sau khi sắp xếp theo khối lượng, băng chuyền sẽ có các viên kẹo với khối lượng lần lượt là: `[5, 5, 5, 10, 10, 15]`.

- Truy vấn $k = 1$: Viên kẹo ở vị trí số 1 nặng 5.
- Truy vấn $k = 4$: Viên kẹo ở vị trí số 4 nặng 10.
- Truy vấn $k = 6$: Viên kẹo ở vị trí số 6 nặng 15.

Bài 5: Định vị số Hamming

Thời gian tối đa: 1.0s
Bộ nhớ tối đa: 256MB

Mô tả bài toán:

Trong lý thuyết số, **số Hamming** (hay còn gọi là số regular) được định nghĩa là các số nguyên dương mà phân tích ra thừa số nguyên tố của chúng chỉ chứa các số $2, 3$ và 5. Nói cách khác, một số Hamming H luôn có dạng $H = 2^x \cdot 3^y \cdot 5^z$ với x, y, z là các số nguyên không âm. (Lưu ý: số 1 cũng được coi là một số Hamming với $x = y = z = 0$).

Hệ thống toán học đã sinh ra một dãy vô hạn tất cả các số Hamming và sắp xếp chúng theo thứ tự tăng dần chặt chẽ (đánh số thứ tự bắt đầu từ 1). Dãy số bắt đầu bằng: $1, 2, 3, 4, 5, 6, 8, 9, 10, 12, 15, \ldots$

Bạn được giao nhiệm vụ xử lý q truy vấn. Mỗi truy vấn cung cấp một số tự nhiên m, hãy xác định xem m nằm ở vị trí thứ bao nhiêu trong dãy số Hamming nói trên.

Input

- Dòng đầu tiên chứa một số nguyên dương q thể hiện số lượng truy vấn.
- q dòng tiếp theo, mỗi dòng chứa một số tự nhiên m cần truy vấn.

Output

- In ra q dòng. Dòng thứ i chứa một số nguyên là vị trí của m trong dãy số Hamming.
- Nếu số m được truy vấn **không phải** là số Hamming, in ra `-1`.

Điều kiện

- $1 \leq q \leq 10^5$.
- $1 \leq m \leq 10^{18}$.

Ví dụ

Input	Output
4 1 5 7 15	1 5 -1 11

Giải thích ví dụ:

- Truy vấn $m = 1$: Vị trí đầu tiên trong dãy.
- Truy vấn $m = 5$: Nằm ở vị trí thứ 5 (dãy là $1, 2, 3, 4, \mathbf{5}, \ldots$).
- Truy vấn $m = 7$: Số 7 là số nguyên tố, không thể biểu diễn dưới dạng $2^x \cdot 3^y \cdot 5^z$, do đó không phải số Hamming. Trả về -1.
- Truy vấn $m = 15$: $15 = 3^1 \cdot 5^1$, nằm ở vị trí thứ 11 trong dãy.

Bài 6: Định vị chuỗi số chia hết (Divisibility Sequence)

Thời gian tối đa: 1.0s
Bộ nhớ tối đa: 256MB

Mô tả bài toán:

Trong quá trình mã hóa một hệ thống cơ sở dữ liệu, các kỹ sư tạo ra một dãy số đặc biệt chứa tất cả các số nguyên dương chia hết cho ít nhất một trong ba số $2, 3$ hoặc 5. Dãy số này được sắp xếp theo thứ tự tăng dần chặt chẽ, bắt đầu với: $2, 3, 4, 5, 6, 8, 9, 10, 12, 14, 15, \ldots$

Để giải mã các gói tin, hệ thống cần truy xuất nhanh giá trị của phần tử nằm ở các vị trí cụ thể trong dãy số trên. Bạn được cung cấp q truy vấn, mỗi truy vấn đưa ra một số nguyên dương k. Nhiệm vụ của bạn là tìm và in ra giá trị của phần tử đứng ở vị trí thứ k trong dãy số.

Input

- Dòng đầu tiên chứa một số nguyên dương q thể hiện số lượng truy vấn.
- q dòng tiếp theo, mỗi dòng chứa một số nguyên dương k là vị trí cần truy vấn.

Output

- In ra q dòng, dòng thứ i chứa một số nguyên duy nhất là kết quả tương ứng cho truy vấn thứ i.

Điều kiện

- $1 \leq q \leq 10^5$.
- $1 \leq k \leq 10^{14}$.

Ví dụ

Input	Output
4 1 6 10 14	2 8 14 20

Giải thích ví dụ:

- Truy vấn $k = 1$: Số đầu tiên trong dãy chia hết cho 2, 3 hoặc 5 là 2.
- Truy vấn $k = 6$: Dãy 6 số đầu tiên là $(2, 3, 4, 5, 6, 8)$. Số ở vị trí thứ 6 là 8.
- Truy vấn $k = 10$: Số ở vị trí thứ 10 là 14.

Bài 7: Thiết lập lõi năng lượng (Max of Min Distance)

Thời gian tối đa: 1.0s
Bộ nhớ tối đa: 256MB

Mô tả bài toán:

Kỹ sư trưởng Leon đang phụ trách thiết kế hệ thống cung cấp năng lượng cho một trạm không gian thế hệ mới. Trên trục lõi của trạm hiện có n khoang chứa trống có thể sử dụng, khoang thứ i nằm ở tọa độ A_i. Leon cần phải lắp đặt k lõi phản ứng lượng tử vào các khoang này để kích hoạt trạm.

Tuy nhiên, các lõi phản ứng liên tục bức xạ nhiệt và từ trường cường độ cao. Nếu đặt hai lõi phản ứng quá gần nhau, hiện tượng cộng hưởng sẽ xảy ra dẫn đến quá tải hệ thống, gây nguy hiểm cho toàn bộ trạm không gian. Do đó, Leon muốn tìm cách chọn ra đúng k khoang chứa để lắp đặt sao cho **khoảng cách ngắn nhất** giữa hai lõi phản ứng bất kỳ (nằm kề nhau) là **lớn nhất** có thể.

Hãy viết chương trình giúp vị kỹ sư trưởng thiết lập hệ thống an toàn nhất.

Input

- Dòng đầu tiên gồm hai số nguyên dương n và k ($k \leq n$).
- Dòng thứ hai gồm n số nguyên A_i biểu diễn tọa độ của các khoang chứa trống. Đảm bảo không có hai khoang nào trùng tọa độ nhau.

Output

- In ra một số nguyên duy nhất là khoảng cách ngắn nhất lớn nhất có thể đạt được giữa hai lõi phản ứng.

Điều kiện

- $2 \leq k \leq n \leq 10^5$.
- $1 \leq A_i \leq 10^9$.

Ví dụ

Input	Output
5 3 1 2 8 4 9	3

Giải thích ví dụ:

- Tọa độ các khoang chứa sau khi sắp xếp lại là: $1, 2, 4, 8, 9$.
- Leon cần đặt $k = 3$ lõi phản ứng.
- Cách đặt tối ưu nhất để tránh quá tải nhiệt là chọn các khoang ở tọa độ: $1, 4, 8$.
- Khoảng cách giữa các lõi liên tiếp lần lượt là: $(4 - 1) = 3$ và $(8 - 4) = 4$.
- Khoảng cách ngắn nhất trong phương án này là 3. Mọi cách lắp đặt khác đều sẽ tạo ra khoảng cách ngắn nhất nhỏ hơn 3, làm tăng nguy cơ cháy nổ.

Bài 8: Lắp đặt trạm phát sóng

Mô tả bài toán:

Trên một trục lộ giao thông thẳng, có N hộ dân cư nằm ở các tọa độ $x_1, x_2, \ldots, x_n$. Chính quyền muốn lắp đặt K trạm phát sóng Wifi. Để đảm bảo công bằng, bán kính phủ sóng của các trạm phải bằng nhau và bằng R. Hãy tìm giá trị R **nhỏ nhất** (nguyên dương) sao cho tất cả N hộ dân đều được phủ sóng.

Input:

- Dòng đầu chứa 2 số nguyên N, K ($1 \leq N \leq 10^5, 1 \leq K \leq N$).
- Dòng thứ hai chứa N số nguyên x_i ($0 \leq x_i \leq 10^9$) đã được sắp xếp tăng dần.

Output:

- Một số nguyên duy nhất là giá trị R nhỏ nhất tìm được.

Ví dụ:

Input	Output
5 2 1 2 5 7 9	2

Giải thích: Với $R = 2$, trạm 1 đặt tại tọa độ 3 phủ sóng [1, 5], trạm 2 đặt tại tọa độ 9 phủ sóng [7, 11]. Tất cả hộ dân (1, 2, 5, 7, 9) đều nằm trong vùng phủ sóng.

Bài 9: Cân bằng hệ thống (Min of Max Sum)

Thời gian tối đa: 1000 ms
Bộ nhớ tối đa: 256 MB

Mô tả bài toán:

Tại trung tâm điều khiển lõi không gian Cyber-Core, kiến trúc sư trưởng Kael đang phải xử lý một luồng dữ liệu khổng lồ. Luồng dữ liệu này bao gồm n gói tin được truyền đến theo một thứ tự cố định, gói tin thứ i có dung lượng là A_i megabyte.

Để đảm bảo an toàn truyền tải và tránh sập nguồn mạng lưới, Kael cần phải cắt luồng dữ liệu này thành **chính xác** k **phân đoạn liên tiếp** để truyền qua k cổng kết nối song song. Tổng dung lượng của một phân đoạn được tính bằng tổng dung lượng của tất cả các gói tin nằm trong phân đoạn đó.

Nhằm tránh hiện tượng "nghẽn cổ chai" (bottleneck), Kael muốn thiết lập cấu hình chia sao cho **mức dung lượng lớn nhất** trong số k phân đoạn đạt giá trị **nhỏ nhất** có thể. Hãy giúp kiến trúc sư Kael tìm ra mức dung lượng tối ưu này (giá trị x).

Input

- Dòng đầu tiên chứa hai số nguyên dương n và k.
- Dòng thứ hai chứa n số nguyên dương $A_1, A_2, \ldots, A_n$ phân tách nhau bởi khoảng trắng.

Output

- In ra một số nguyên duy nhất là giá trị x tối ưu (dung lượng lớn nhất trong k phân đoạn là nhỏ nhất có thể).

Điều kiện

- $1 \leq k \leq n \leq 10^5$.
- $1 \leq A_i \leq 10^9$.

Ví dụ

Input	Output
4 2 10 20 30 40	60

Giải thích ví dụ:

- Kael có $n = 4$ gói tin và cần chia thành $k = 2$ phân đoạn liên tiếp. Có các cách chia sau:
- Cách 1: (10) và $(20, 30, 40) \Rightarrow$ Dung lượng các phân đoạn là 10 và 90. Giá trị lớn nhất là 90.

- Cách 2: $(10, 20)$ và $(30, 40) \Rightarrow$ Dung lượng các phân đoạn là 30 và 70. Giá trị lớn nhất là 70.
- Cách 3: $(10, 20, 30)$ và $(40) \Rightarrow$ Dung lượng các phân đoạn là 60 và 40. Giá trị lớn nhất là 60.
- So sánh các phương án, giá trị lớn nhất nhỏ nhất có thể đạt được là 60 (ở Cách 3).

Bài 10: Bảng cửu chương (CSES - Multiplication Table)

Thời gian tối đa: 1.0s
Bộ nhớ tối đa: 512MB

Mô tả bài toán:

Tìm phần tử giữa khi các số trong bảng cửu chương $n \times n$ được sắp xếp theo thứ tự tăng dần với n là số lẻ.

Ví dụ, bảng cửu chương 3×3 được biểu diễn như sau:

1	2	3
2	4	6
3	6	9

Các số theo thứ tự tăng dần là $[1, 2, 2, 3, 3, 4, 6, 6, 9]$, như vậy kết quả là 3.

Input

- Dòng đầu vào duy nhất có số nguyên n ($1 \leq n < 10^6$).

Output

- In một số nguyên: câu trả lời cho nhiệm vụ.

Ví dụ

Input	Output
3	3

Bài 11: Dãy con có trung bình cộng lớn nhất (Maximum Average Subarray)

Thời gian tối đa: 1.0s
Bộ nhớ tối đa: 256MB

Mô tả bài toán:

Cho một dãy số A gồm n phần tử $A_1, A_2, \ldots, A_n$. Hãy tìm một dãy con gồm các phần tử liên tiếp của A có độ dài **ít nhất là** k sao cho trung bình cộng của các phần tử trong dãy con đó là lớn nhất có thể.

Input

- Dòng đầu tiên chứa hai số nguyên dương n và k.
- Dòng thứ hai chứa n số nguyên $A_1, A_2, \ldots, A_n$ phân tách nhau bởi khoảng trắng.

Output

- In ra một số thực duy nhất là giá trị trung bình cộng lớn nhất tìm được. Kết quả được chấp nhận nếu sai số so với đáp án chuẩn không vượt quá 10^{-3} (khuyên dùng in ra với 3 chữ số thập phân).

Điều kiện

- $1 \leq k \leq n \leq 10^5$.
- $-10^5 \leq A_i \leq 10^5$.

Ví dụ

Input	Output
5 2 1 2 5 7 2	6.000

Giải thích ví dụ:

- Ta cần tìm dãy con liên tiếp có độ dài ít nhất là 2.
- Chọn dãy con từ vị trí thứ 3 đến vị trí thứ 4 là $(5, 7)$.
- Độ dài dãy con là $2 \geq 2$.
- Trung bình cộng là $(5 + 7)/2 = 6.000$. Đây là giá trị lớn nhất có thể đạt được.

Bài 12: Hàng rào năng lượng (FENCE2)

Thời gian tối đa: 1.0s
Bộ nhớ tối đa: 256MB

Mô tả bài toán:

Sau sự cố tại trung tâm điều khiển, Kỹ sư trưởng Leon tiếp tục nhận nhiệm vụ gia cố "Hàng rào năng lượng" để bảo vệ căn cứ khỏi các đợt bão từ. Hàng rào được chia làm n phân đoạn, đoạn thứ i hiện đang có mức năng lượng là a_i.

Leon mang theo một xe kéo chứa m lõi pin lượng tử dự phòng, lõi thứ j có mức năng lượng b_j. Các lõi pin được xếp chồng lên nhau, lõi b_1 nằm trên cùng.

Leon đi dọc theo hàng rào từ phân đoạn 1 đến n. Nếu muốn tăng cường năng lượng cho một phân đoạn, anh ta có thể lấy một lõi pin trên xe gắn vào. Khi đó, mức năng lượng mới của phân đoạn sẽ bằng năng lượng cũ cộng với năng lượng của lõi pin $(a_i + b_j)$. Để đảm bảo cấu trúc, **không được gắn quá 1 lõi pin vào 1 phân đoạn rào**.

Quy trình sử dụng pin của Leon diễn ra như sau: Tại mỗi phân đoạn, anh ta có thể lấy lõi pin trên cùng để sử dụng, hoặc vứt bỏ một vài lõi pin trên cùng để lấy lõi tiếp theo. Vì bộ đồ bảo hộ rất nặng, Leon chỉ tiến tới phía trước, không thể đi lùi và **không thể nhặt lại những lõi pin đã vứt đi**.

Năng lượng của toàn bộ hàng rào được đánh giá bằng **mức năng lượng của phân đoạn yếu nhất**. Hãy giúp Leon tính toán xem có thể thiết lập hàng rào đạt được mức năng lượng lớn nhất là bao nhiêu.

Input

- Dòng đầu tiên chứa số nguyên n - số đoạn hàng rào.
- Dòng thứ hai chứa n số nguyên $a_1, a_2, \ldots, a_n$ - năng lượng hiện tại của các đoạn rào, theo đúng thứ tự Leon sẽ đi qua.
- Dòng thứ ba chứa số nguyên m - số lượng lõi pin dự phòng.
- Dòng thứ tư chứa m số nguyên $b_1, b_2, \ldots, b_m$ - mức năng lượng của các lõi pin, tính từ trên cùng xuống dưới.

Output

- In ra một dòng duy nhất là mức năng lượng lớn nhất có thể của hàng rào sau khi gia cố.

Điều kiện

- $1 \leq n, m \leq 10^5$.
- $1 \leq a_i, b_j \leq 10^8$.

Ví dụ

Input	Output
3 1 5 6 5 2 5 4 1 7	6

Giải thích ví dụ:

- Ta có thể đạt được ngưỡng năng lượng tối thiểu là 6 cho toàn bộ hàng rào bằng chiến thuật sau:
- **Đoạn 1 ($a_1 = 1$):** Cần tối thiểu 5 năng lượng. Bỏ qua lõi pin đầu tiên (2), lấy lõi pin tiếp theo (5). Năng lượng đoạn 1 trở thành $1 + 5 = 6$. Các lõi pin còn lại trên xe: $[4, 1, 7]$.
- **Đoạn 2 ($a_2 = 5$):** Cần tối thiểu 1 năng lượng. Lấy ngay lõi pin trên cùng (4). Năng lượng đoạn 2 trở thành $5 + 4 = 9$. Các lõi pin còn lại trên xe: $[1, 7]$.
- **Đoạn 3 ($a_3 = 6$):** Dã đạt mức 6, không cần dùng thêm pin.
- Tất cả các đoạn đều có năng lượng ≥ 6. Không có cách nào dùng số pin trên để kéo phân đoạn yếu nhất lên mức 7 (bạn có thể tự kiểm chứng). Do đó đáp án là 6.

Bài 13: Xưởng sản xuất đèn lồng (LANTERN)

Thời gian tối đa: 1.0s
Bộ nhớ tối đa: 256MB

Mô tả bài toán:

Nhân dịp lễ hội Tết Trung thu sắp tới, bạn được giao trọng trách quản lý một xưởng sản xuất lồng đèn quy mô lớn. Xưởng nhận được đơn hàng chế tạo m loại lồng đèn khác nhau, trong đó loại thứ i cần phải sản xuất đúng a_i chiếc.

Để phục vụ sản xuất, xưởng được trang bị một hệ thống gồm n robot tự động. Bạn là người trực tiếp phân công nhiệm vụ cho các robot này với những quy tắc sau:

- Mỗi robot chỉ được phép lắp ráp **duy nhất một loại** lồng đèn trong suốt quá trình làm việc (bạn có quyền quyết định phân công robot nào làm loại nào).
- Thời gian để bất kỳ robot nào lắp ráp xong một chiếc lồng đèn (bất kể loại nào) đều là 1 đơn vị thời gian.
- Bạn không bắt buộc phải sử dụng toàn bộ n robot nếu không cần thiết.

Vì tất cả các robot được kích hoạt làm việc đồng thời, nên tổng thời gian để hoàn thành toàn bộ đơn hàng sẽ bằng thời gian hoạt động của con robot làm việc lâu nhất.

Hãy tìm phương án phân công robot sao cho thời gian để xưởng hoàn thành tất cả các lồng đèn là **ít nhất có thể**.

Input

- Dòng đầu tiên chứa hai số nguyên dương n và m (n là số robot, m là số loại lồng đèn).
- Dòng thứ hai chứa m số nguyên dương $a_1, a_2, \ldots, a_m$ biểu diễn số lượng lồng đèn cần sản xuất cho từng loại.

Output

- In ra một số nguyên duy nhất là thời gian hoàn thành đơn hàng tối thiểu.

Điều kiện

- $1 \leq n \leq 10^9$.
- $1 \leq m \leq 10^5$.
- $m \leq n$.
- $1 \leq a_i \leq 10^9$.

Ví dụ

Input	Output
5 2 7 4	3

Giải thích ví dụ:

- Có $n = 5$ robot và $m = 2$ loại lồng đèn. Cần sản xuất 7 chiếc loại 1 và 4 chiếc loại 2.
- Phân công tối ưu:
  - Cử 3 robot làm lồng đèn loại 1. Khối lượng công việc chia ra là: 3 chiếc, 2 chiếc, 2 chiếc (robot làm lâu nhất mất 3 đơn vị thời gian).
  - Cử 2 robot làm lồng đèn loại 2. Khối lượng công việc chia ra là: 2 chiếc, 2 chiếc (robot làm lâu nhất mất 2 đơn vị thời gian).
- Thời gian hoàn thành tổng thể là $\max(3, 2) = 3$ đơn vị thời gian.

Bài 14: Dung hợp Lõi Năng Lượng

Thời gian tối đa: 1.0s
Bộ nhớ tối đa: 256MB

Mô tả bài toán:

Sau khi thiết lập xong hàng rào, Kỹ sư trưởng Leon thu thập được một mảng gồm n lõi năng lượng tàn dư. Lõi thứ i hiện đang có mức năng lượng là a_i.

Để tối ưu hóa sức mạnh cho các vũ khí phòng thủ, Leon có thể thực hiện một thao tác **dung hợp** như sau:

- Chọn ra hai lõi năng lượng bất kỳ là a_i và a_j ($i \neq j$).

- Nếu lõi a_i có mức năng lượng lớn hơn hoặc bằng lõi a_j ($a_i \geq a_j$), lõi a_i có thể hấp thụ hoàn toàn lõi a_j.
- Hậu quả của thao tác này là lõi a_j bị phá hủy (xóa khỏi mảng), còn mức năng lượng của lõi a_i sẽ được **tăng thêm đúng 1 đơn vị** ($a_i = a_i + 1$).

Leon có thể thực hiện thao tác dung hợp này bao nhiêu lần tùy ý. Hệ thống chỉ huy đưa ra q kịch bản phòng thủ (truy vấn). Trong kịch bản thứ i, hệ thống yêu cầu mức năng lượng tối thiểu để vũ khí hoạt động là k_i.

Nhiệm vụ của bạn là tính toán xem, với mỗi kịch bản k_i, Leon có thể tạo ra **nhiều nhất bao nhiêu** lõi năng lượng có mức năng lượng $\geq k_i$ sau khi thực hiện các phép dung hợp.

Input

- Dòng đầu tiên chứa hai số nguyên dương n và q (n là số lõi năng lượng, q là số kịch bản truy vấn).
- Dòng thứ hai chứa n số nguyên dương $a_1, a_2, \ldots, a_n$ biểu diễn mức năng lượng ban đầu của các lõi.
- q dòng tiếp theo, dòng thứ i chứa một số nguyên dương k_i biểu diễn yêu cầu năng lượng của kịch bản thứ i.

Output

- In ra q dòng. Dòng thứ i là số lượng tối đa các lõi đạt mức năng lượng $\geq k_i$ trong kịch bản thứ i.

Điều kiện

- $1 \leq n, q \leq 10^5$.
- $1 \leq a_i \leq 10^9$.
- $1 \leq k_i \leq 10^9$.

Ví dụ

Input	Output
5 2 21 9 5 8 10 10 15	3 1
5 1 1 2 3 4 5 100	0

Giải thích ví dụ:

- **Test thứ nhất:**
  - Truy vấn 1 ($k = 10$): Lõi số 2 (giá trị 9) có thể hấp thụ lõi số 4 (giá trị 8) vì $9 \geq 8$. Sau dung hợp, lõi số 2 tăng lên $9 + 1 = 10$, và lõi 8 biến mất. Mảng trở thành $(21, 10, 5, 10)$. Lúc này có 3 phần tử ≥ 10 (là 21, 10 và 10). Đây là phương án tối ưu.
  - Truy vấn 2 ($k = 15$): Dù thực hiện dung hợp cách nào đi chăng nữa, cũng chỉ có duy nhất lõi 21 là duy trì được mức năng lượng ≥ 15. Kết quả là 1.
- **Test thứ hai:**
  - Truy vấn 1 ($k = 100$): Lõi lớn nhất ban đầu chỉ là 5, dù hấp thụ toàn bộ các lõi còn lại thì mức năng lượng tối đa cũng chỉ lên được $5 + 4 = 9$, không thể nào tạo ra được lõi ≥ 100. Kết quả là 0.

Bài 15: Phân bổ năng lượng 3 pha

Thời gian tối đa: 1.0s
Bộ nhớ tối đa: 256MB

Mô tả bài toán:

Trung tâm điều khiển lõi không gian đang đối mặt với nguy cơ quá tải. Kỹ sư trưởng Leon cần nạp năng lượng cho một lò phản ứng lượng tử hoạt động theo chu trình 3 pha.

Trong tay Leon hiện có N khối năng lượng được xếp thành một hàng dọc, đánh số từ 1 đến N. Khối thứ i mang mức năng lượng là A_i. Để nạp vào lò phản ứng, Leon bắt buộc phải cắt hàng khối năng lượng này thành đúng 3 đoạn liên tiếp nhau:

- **Pha 1:** Gồm các khối từ 1 đến i. Tổng năng lượng là $x = A_1 + A_2 + \cdots + A_i$.
- **Pha 2:** Gồm các khối từ $i+1$ đến j. Tổng năng lượng là $y = A_{i+1} + A_{i+2} + \cdots + A_j$.
- **Pha 3:** Gồm các khối từ $j+1$ đến N. Tổng năng lượng là $z = A_{j+1} + A_{j+2} + \cdots + A_N$.

(Yêu cầu: $1 \leq i < j < N$ để đảm bảo pha nào cũng có ít nhất 1 khối năng lượng).

Sự ổn định của lò phản ứng phụ thuộc vào độ chênh lệch năng lượng giữa 3 pha. Leon cần tìm cách phân chia sao cho độ lệch giữa pha nhận nhiều năng lượng nhất và pha nhận ít năng lượng nhất là **nhỏ nhất có thể**.

Gọi $T = \max(x, y, z) - \min(x, y, z)$. Hãy giúp Leon tìm giá trị nhỏ nhất của T.

Input

- Dòng thứ nhất chứa số nguyên dương N là số khối năng lượng.
- Dòng thứ hai chứa N số nguyên dương $A_1, A_2, \ldots, A_N$ phân tách nhau bởi dấu cách.

Output

- In ra một số nguyên duy nhất là giá trị nhỏ nhất của T.

Điều kiện

- $3 \leq N \leq 10^5$.
- $1 \leq A_i \leq 10^3$.

Ví dụ

Input	Output
9 1 1 3 2 1 3 2 1 1	2

Giải thích ví dụ:

- Leon có $N = 9$ khối năng lượng.
- Cách chia tối ưu nhất là chọn $i = 3$ và $j = 6$:
  - Pha 1 (khối 1 đến 3): $x = 1 + 1 + 3 = 5$.
  - Pha 2 (khối 4 đến 6): $y = 2 + 1 + 3 = 6$.
  - Pha 3 (khối 7 đến 9): $z = 2 + 1 + 1 = 4$.
- Pha lớn nhất là 6, pha nhỏ nhất là 4. Độ chênh lệch $T = 6 - 4 = 2$.
- Không có cách chia nào mang lại độ chênh lệch $T < 2$.

Bài 16: Khai thác tinh thể (SAW - Chặt Cây)

Thời gian tối đa: 1.0s
Bộ nhớ tối đa: 256MB

Mô tả bài toán:

Để sửa chữa bộ vi mạch lõi của phi thuyền, Kỹ sư Leon cần thu thập ít nhất M đơn vị tinh thể năng lượng. Anh phát hiện ra một khu rừng pha lê nguyên thủy với N trụ tinh thể khổng lồ. Trụ thứ i có chiều cao là A_i.

Leon sở hữu một phi thuyền khai thác được trang bị lưỡi cắt laser siêu dẹt. Lưỡi laser hoạt động như sau: Leon thiết lập một tham số độ cao là H, sau đó phi thuyền sẽ bay ngang qua khu rừng và cắt bỏ toàn bộ phần ngọn của các trụ tinh thể vượt quá độ cao H. Những trụ có chiều cao $\leq H$ sẽ không bị ảnh hưởng. Phần tinh thể bị cắt rơi xuống sẽ được phi thuyền thu thập.

Ví dụ: Nếu khu rừng có các trụ cao $20, 15, 10, 17$ và Leon đặt độ cao laser $H = 15$, chiều cao còn lại của các trụ lần lượt là $15, 15, 10, 15$. Số tinh thể Leon thu thập được là $5 + 0 + 0 + 2 = 7$ đơn vị.

Nhiệm vụ của bạn là giúp Kỹ sư Leon xác định độ cao H **lớn nhất** có thể thiết lập sao cho phi thuyền vẫn thu thập được **ít nhất** M đơn vị tinh thể (để tiết kiệm tối đa năng lượng cho phi thuyền và không hủy hoại môi trường quá mức).

Input

- Dòng đầu tiên gồm hai số nguyên N (số lượng trụ tinh thể) và M (lượng tinh thể cần thiết).
- Dòng thứ hai chứa N số nguyên $A_1, A_2, \ldots, A_N$ biểu diễn chiều cao của các trụ tinh thể. Dữ liệu luôn đảm bảo tổng chiều cao của tất cả các trụ lớn hơn hoặc bằng M (Leon luôn có cách hoàn thành nhiệm vụ).

Output

- In ra một số nguyên duy nhất là độ cao H lớn nhất thỏa mãn yêu cầu.

Điều kiện

- $1 \leq N \leq 10^6$.
- $1 \leq M \leq 2 \times 10^9$.
- $0 \leq A_i \leq 10^9$.

Ví dụ

Input	Output
4 7 20 15 10 17	15
5 20 4 42 40 26 46	36

Giải thích ví dụ 2:

- Chiều cao các trụ là $4, 42, 40, 26, 46$.
- Leon cần ít nhất $M = 20$ đơn vị tinh thể.
- Nếu Leon đặt $H = 36$, các phần bị cắt ở mỗi trụ lần lượt là:

- Trụ 1 (cao 4): Không bị cắt (0)
- Trụ 2 (cao 42): Cắt được $42 - 36 = 6$
- Trụ 3 (cao 40): Cắt được $40 - 36 = 4$
- Trụ 4 (cao 26): Không bị cắt (0)
- Trụ 5 (cao 46): Cắt được $46 - 36 = 10$

- Tổng cộng thu được $0 + 6 + 4 + 0 + 10 = 20$ đơn vị, vừa đúng lượng cần thiết. Độ cao $H = 36$ là mức lớn nhất khả thi.

Bài 17: Quét phổ tần số (SUBMATRIX - Ma trận con)

Thời gian tối đa: 1.0s
Bộ nhớ tối đa: 256MB

Mô tả bài toán:

Hệ thống ra-đa của phi thuyền do Kỹ sư Leon điều khiển vừa quét được một khu vực không gian hai chiều có kích thước $M \times N$. Tại mỗi tọa độ (i, j) trong khu vực này, ra-đa đo được một chỉ số tần số bức xạ tự nhiên, được biểu diễn bằng một số tự nhiên.

Để thiết lập một "Vùng an toàn lượng tử", tương đương với một ma trận vuông con trong khu vực không gian quét được, Leon cần đảm bảo rằng số lượng các dải tần số bức xạ phân biệt (các giá trị khác nhau) bên trong Vùng an toàn này **không được vượt quá** K để tránh làm nhiễu hệ thống.

Nhiệm vụ của bạn là hãy giúp Leon xác định độ dài cạnh lớn nhất có thể của "Vùng an toàn lượng tử" (ma trận vuông con lớn nhất) thỏa mãn điều kiện trên.

Input

- Dòng đầu tiên chứa ba số tự nhiên M, N và K.
- M dòng tiếp theo, mỗi dòng chứa N số tự nhiên mô tả các chỉ số tần số bức xạ tại từng tọa độ trong ma trận.

Output

- In ra một số nguyên duy nhất là độ dài cạnh của "Vùng an toàn lượng tử" lớn nhất tìm được.

Điều kiện

- $1 \leq K \leq M \times N$.
- Các chỉ số tần số bức xạ (phần tử của ma trận) không vượt quá $M \times N$.
- **Subtask 1 (30 điểm):** $1 \leq M, N \leq 30$.
- **Subtask 2 (40 điểm):** $1 \leq M, N \leq 150$.
- **Subtask 3 (30 điểm):** $1 \leq M, N \leq 300$.

Ví dụ

Input	Output
5 7 3 6 5 7 3 6 6 7 5 7 5 5 7 3 7 3 3 5 3 5 6 7 7 7 5 5 5 6 7 7 7 6 5 6 3 5	3

Bài 18: Đối chiếu tín hiệu lượng tử

Thời gian tối đa: 1.0s
Bộ nhớ tối đa: 256MB

Mô tả bài toán:

Trong quá trình giải mã hai lõi dữ liệu ngoài hành tinh, Kỹ sư Leon thu xuất được hai chuỗi tín hiệu năng lượng: Chuỗi Alpha gồm m chỉ số $a_1, a_2, \ldots, a_m$ và Chuỗi Beta gồm n chỉ số $b_1, b_2, \ldots, b_n$.

Để đánh giá khả năng dung hợp của hai lõi, hệ thống AI cần so sánh mức năng lượng của mọi **dải tín hiệu con liên tiếp** thuộc chuỗi Alpha với mọi **dải tín hiệu con liên tiếp** thuộc chuỗi Beta. Cụ thể, hệ thống xét các bộ 4 chỉ số (x, y, u, v) thỏa mãn $1 \leq x \leq y \leq m$ và $1 \leq u \leq v \leq n$, sau đó phân loại chúng thành 3 tập hợp:

- **Tập S (Smaller):** Số lượng bộ chỉ số mà tổng năng lượng dải Alpha nhỏ hơn dải Beta ($\sum_{i=x}^{y} a_i < \sum_{i=u}^{v} b_i$).
- **Tập E (Equal):** Số lượng bộ chỉ số mà tổng năng lượng dải Alpha cộng hưởng hoàn hảo với dải Beta ($\sum_{i=x}^{y} a_i = \sum_{i=u}^{v} b_i$).
- **Tập G (Greater):** Số lượng bộ chỉ số mà tổng năng lượng dải Alpha áp đảo dải Beta ($\sum_{i=x}^{y} a_i > \sum_{i=u}^{v} b_i$).

Nhiệm vụ của bạn là lập trình giúp Kỹ sư Leon tính chính xác số lượng phần tử của 3 tập hợp S, E và G.

Input

- Dòng đầu tiên gồm 2 số nguyên dương m, n.
- Dòng thứ hai gồm m số nguyên $a_1, a_2, \ldots, a_m$ biểu diễn chuỗi tín hiệu Alpha.
- Dòng thứ ba gồm n số nguyên $b_1, b_2, \ldots, b_n$ biểu diễn chuỗi tín hiệu Beta.

Output

- In ra duy nhất một dòng chứa 3 số nguyên lần lượt là kết quả S, E và G, cách nhau bởi khoảng trắng.

Điều kiện

- $1 \leq m, n \leq 1000$.
- Giá trị tuyệt đối của các phần tử a_i, b_j không vượt quá 10^9 ($|a_i|, |b_j| \leq 10^9$).

Ví dụ

Input	Output
1 2 0 1 -1	1 1 1

Giải thích ví dụ:

- Chuỗi Alpha có độ dài $m = 1$, dải con duy nhất là [0] với tổng bằng 0.
- Chuỗi Beta có độ dài $n = 2$, gồm các dải con liên tiếp: [1] (tổng 1), [−1] (tổng −1), và [1, −1] (tổng 0).

- So sánh dải Alpha với các dải Beta:
  - $0 < 1 \implies$ thuộc tập S (đếm được 1).
  - $0 = 0 \implies$ thuộc tập E (đếm được 1).
  - $0 > -1 \implies$ thuộc tập G (đếm được 1).
- Do đó, kết quả là `1 1 1`.

Ví dụ

Input	Output
3 4 1 -2 3 2 -1 0 1	22 12 26

Bài 19: Dây chuyền cơ giáp (Factory Machines / Gà đẻ trứng)

Thời gian tối đa: 1.0s
Bộ nhớ tối đa: 256MB

Mô tả bài toán:

Căn cứ Cyber-Core đang bị đe dọa, Kỹ sư trưởng Leon cần sản xuất khẩn cấp M đơn vị cơ giáp tự hành để gia cố tuyến phòng thủ.

Trong xưởng chế tạo hiện có N máy in 3D lượng tử. Các máy này có công suất khác nhau: Máy thứ i cần chính xác T_i giây để in hoàn chỉnh một đơn vị cơ giáp. Leon có thể kích hoạt toàn bộ N máy in hoạt động đồng thời và hoàn toàn độc lập với nhau.

Hãy giúp Leon tính toán xem: Kể từ lúc bấm nút khởi động đồng loạt, cần **ít nhất bao nhiêu giây** để toàn bộ xưởng in ra đủ (hoặc dư) M đơn vị cơ giáp?

Input

- Dòng đầu tiên chứa hai số nguyên dương N và M (N là số lượng máy in, M là số lượng cơ giáp cần sản xuất).
- Dòng thứ hai chứa N số nguyên dương $T_1, T_2, \ldots, T_N$ phân tách nhau bởi khoảng trắng, biểu diễn thời gian in 1 cơ giáp của từng máy.

Output

- In ra một số nguyên duy nhất là thời gian ngắn nhất (tính bằng giây) để sản xuất đủ số lượng yêu cầu.

Điều kiện

- $1 \leq N \leq 10^5$.
- $1 \leq M \leq 10^9$.
- $1 \leq T_i \leq 10^9$.

Ví dụ

Input	Output
3 7 3 2 5	8

Giải thích ví dụ:

- Xưởng có $N = 3$ máy in với tốc độ in lần lượt là 3 giây/cái, 2 giây/cái và 5 giây/cái. Cần in $M = 7$ cơ giáp.
- Nếu Leon cho xưởng chạy trong **8 giây**:
  - Máy 1 (3s/cái): In được $\lfloor 8/3 \rfloor = 2$ cái.
  - Máy 2 (2s/cái): In được $\lfloor 8/2 \rfloor = 4$ cái.
  - Máy 3 (5s/cái): In được $\lfloor 8/5 \rfloor = 1$ cái.
- Tổng số cơ giáp thu được sau 8 giây là $2 + 4 + 1 = 7$ cái (Vừa đủ yêu cầu).
- Nếu chỉ chạy 7 giây, tổng sản phẩm sẽ là $2 + 3 + 1 = 6$ cái (Không đủ). Do đó 8 giây là khoảng thời gian tối ưu nhất.

Bài 20: Tổng M cặp lớn nhất

Thời gian tối đa: 1.0s
Bộ nhớ tối đa: 256MB

Mô tả bài toán:

Cho một mảng A gồm N số nguyên dương $A_1, A_2, \ldots, A_N$.

Xét tất cả N^2 cặp chỉ số (i, j) với $1 \leq i, j \leq N$. Trọng số của mỗi cặp (i, j) được định nghĩa là $A_i + A_j$. Lưu ý rằng các cặp (i, j) và (j, i) được coi là hai cặp khác nhau nếu $i \neq j$.

Nhiệm vụ của bạn là chọn ra đúng M cặp có trọng số lớn nhất. Hãy tính và in ra tổng trọng số của M cặp được chọn này.

Input

- Dòng đầu tiên gồm hai số nguyên dương N và M.
- Dòng thứ hai gồm N số nguyên dương $A_1, A_2, \ldots, A_N$.

Output

- In ra một số nguyên duy nhất là tổng trọng số của M cặp lớn nhất.

Điều kiện

- $1 \leq N \leq 10^5$.
- $1 \leq M \leq N^2$.
- $1 \leq A_i \leq 10^5$.

Ví dụ

Input	Output
3 4 1 2 3	20

Giải thích ví dụ:

- Có $N = 3$ phần tử, tạo ra $3^2 = 9$ cặp.
- Trọng số của các cặp được sắp xếp giảm dần như sau:
  - Cặp $(3, 3) \Rightarrow$ Tổng $= 3 + 3 = 6$
  - Cặp $(2, 3) \Rightarrow$ Tổng $= 2 + 3 = 5$

- Cặp $(3, 2) \Rightarrow$ Tổng $= 3 + 2 = 5$
- Cặp $(2, 2) \Rightarrow$ Tổng $= 2 + 2 = 4$
- Cặp $(1, 3) \Rightarrow$ Tổng $= 1 + 3 = 4$
- Cặp $(3, 1) \Rightarrow$ Tổng $= 3 + 1 = 4$
- Cặp $(1, 2) \Rightarrow$ Tổng $= 1 + 2 = 3$
- Cặp $(2, 1) \Rightarrow$ Tổng $= 2 + 1 = 3$
- Cặp $(1, 1) \Rightarrow$ Tổng $= 1 + 1 = 2$

- Cần chọn $M = 4$ cặp lớn nhất. Các giá trị đó là $6, 5, 5$ và một số 4.
- Tổng cần tìm: $6 + 5 + 5 + 4 = 20$.

Ví dụ 2

Input	**Output**
3 4 2 3 5	33

Giải thích ví dụ 2:

- Có $N = 3$ phần tử, tạo ra $3^2 = 9$ cặp.
- Trọng số của các cặp được sắp xếp giảm dần như sau:
  - Cặp $(3, 3) \Rightarrow$ Tổng $= 5 + 5 = 10$
  - Cặp $(2, 3) \Rightarrow$ Tổng $= 3 + 5 = 8$
  - Cặp $(3, 2) \Rightarrow$ Tổng $= 5 + 3 = 8$
  - Cặp $(1, 3) \Rightarrow$ Tổng $= 2 + 5 = 7$
  - Cặp $(3, 1) \Rightarrow$ Tổng $= 5 + 2 = 7$
  - Cặp $(2, 2) \Rightarrow$ Tổng $= 3 + 3 = 6$
  - Cặp $(1, 2) \Rightarrow$ Tổng $= 2 + 3 = 5$
  - Cặp $(2, 1) \Rightarrow$ Tổng $= 3 + 2 = 5$
  - Cặp $(1, 1) \Rightarrow$ Tổng $= 2 + 2 = 4$
- Ta cần chọn $M = 4$ cặp lớn nhất. Các giá trị đó là $10, 8, 8$ và đúng một số 7.
- (Lưu ý: Mặc dù vị trí thứ 5 cũng có giá trị là 7, nhưng ta chỉ lấy đúng 4 phần tử đầu tiên).
- Tổng cần tìm: $10 + 8 + 8 + 7 = 33$.